CE-355 Lab 6
VR360: 360° Panorama Image Viewer

1. Introduction

A 360° panorama image is a photograph that captures the view in every direction from a single point: all the way around, straight up and straight down. A virtual reality viewer lets you look around inside one: as you turn your head, the view on the screen turns with you. In this lab, you will build that viewer on an FPGA. A 360° panorama is stored in the board's external SDRAM. An IMU (inertial measurement unit) sensor reports which way it is pointing, and the FPGA renders the part of the panorama you are "looking at" in that direction, then displays it on an HDMI monitor. Turn the sensor, and the view turns.

Rendering the view is a real computing problem. For every one of the 307,200 pixels on the screen, the design must work out which direction that pixel looks in, rotate that direction to match the sensor's orientation, convert the rotated direction into a position in the panorama, and fetch that pixel from SDRAM. The design does this with a chain of hardware modules, each performing one step of the math. The modules pass data down the chain using a two-way **valid/ready handshake**, so each one can take as many clock cycles as it needs.

This lab introduces several new VHDL and RTL concepts:

- **Two-process state machines:** separating a design into a clocked process that holds the registers and a combinational FSM process that computes their next values.
- **Valid/ready handshakes:** connecting modules that each take a different and variable number of clock cycles, so that data is never lost or duplicated.
- **Fixed-point trigonometry:** computing sines, cosines and other functions with lookup tables, and multiplying fixed-point fractions.
- **3D math in hardware:** building a rotation matrix and multiplying it by a vector.
- **External memory:** reading data from an SDRAM chip through a memory controller, and waiting for it to respond.
- **Frame buffers:** using on-chip block RAM to decouple a renderer from the display.

It also introduces new parts of an FPGA system:

- **A serial sensor:** the 9-axis IMU module, which streams its orientation over a UART serial link.
- **Loading data over JTAG:** sending a large image from the computer into the board's SDRAM through the programming cable, after the FPGA is configured.

The IMU interface, the SDRAM controller, the JTAG image loader, the conversion from direction to angles, the HDMI output and the top-level system are provided for you. You will complete

the rotation matrix, the ray generator, the matrix-vector multiplier, the image reader, the frame reader and the seven-segment display. You will then simulate the design on three panoramas, and synthesize, program and test it on the DE25-Standard with the IMU.

By completing this lab, you will:

- Write multi-cycle state machines in the two-process style.
- Use a valid/ready handshake to connect a chain of modules.
- Use lookup tables and fixed-point multiplication to compute trigonometric and matrix operations.
- Explain how a 3D rotation and an equirectangular projection turn a sensor orientation into an image.
- Read from an external SDRAM through a controller.
- Use a frame buffer to connect a variable-speed renderer to a fixed-rate display.
- Connect a UART sensor module to the FPGA and interpret its data.
- Simulate, program and test a real-time FPGA rendering system.

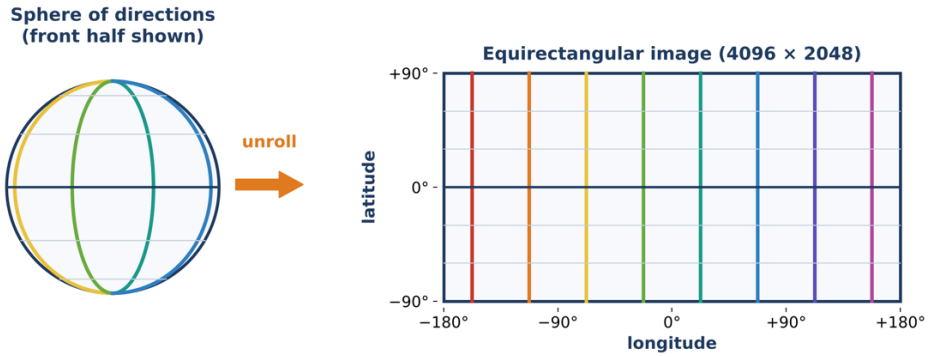
2. Background: Viewing a 360° Panorama

2.1 Equirectangular Panoramas

Imagine standing at the center of a large sphere, with the scene painted on the inside of it. Every direction you can look in hits one point on the sphere. A 360° panorama is a record of that sphere.

To store the sphere as an ordinary rectangular image, it is unrolled the same way a world map unrolls the globe. Each direction is described by two angles:

- **Longitude:** how far left or right you are looking, from −180° to +180°, with 0° straight ahead.
- **Latitude:** how far up or down you are looking, from −90° (straight down) to +90° (straight up), with 0° at the horizon.



An equirectangular panorama with longitude (−180° to +180°) marked along the bottom and latitude (+90° to −90°) along the side. Each meridian (vertical circle) becomes a vertical line. Each parallel (horizontal ring) becomes a horizontal line.

In the image, longitude runs across from left to right, and latitude runs from top (+90°) to bottom (−90°). This layout is called an **equirectangular projection**. It is always twice as wide as it is tall, because longitude covers 360° and latitude covers 180°. The panoramas in this lab are 4096 × 2048 pixels on the board.

The top and bottom rows of the image are stretched across the full width, even though they represent single points (straight up and straight down). This is why equirectangular images look distorted near the top and bottom. The viewer you build undoes this distortion: it shows the scene as a normal camera would see it.

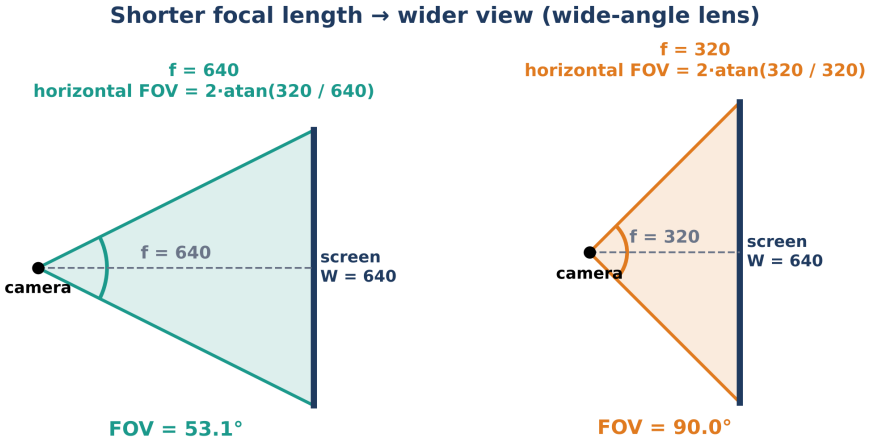
2.2 The Virtual Camera and Rays

The viewer works like a camera at the center of the sphere. Picture the screen as a flat window held in front of the camera, at a distance called the **focal length**. Each pixel on the screen is a small square of that window, and the camera looks at the scene through it. The line from the camera through a pixel is that pixel's **ray**.

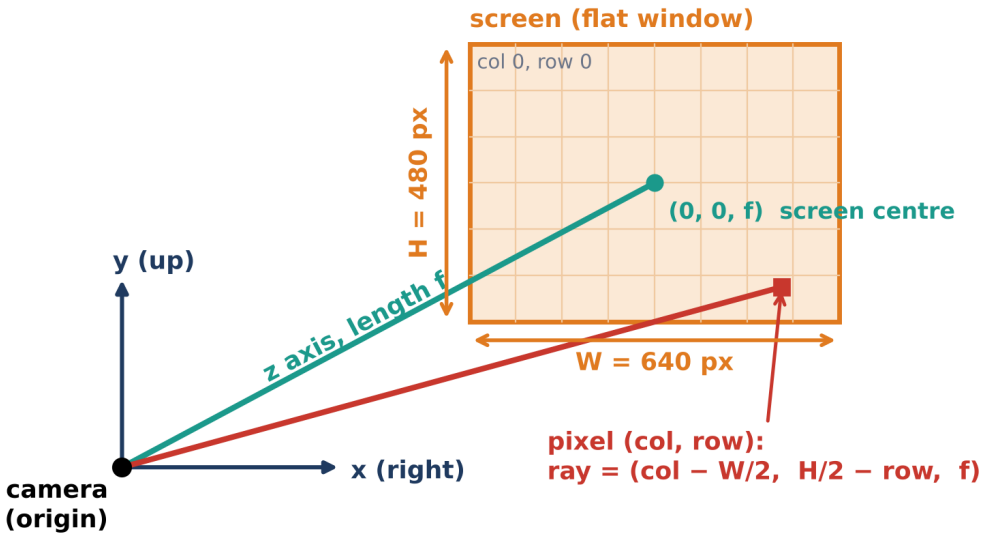
Set up 3D coordinates with the camera at the origin: **x** pointing right, **y** pointing up, and **z** pointing straight ahead out of the screen. For the pixel in column *col* and row *row* of a screen that is W pixels wide and H pixels tall, the ray points in the direction

$$x = col - W/2$$
$$y = H/2 - row$$
$$z = f$$

where *f* is the focal length in pixels. The center of the screen gives the ray (0, 0, *f*), straight ahead. Pixels to the right have positive x, and pixels near the top have positive y (row numbers increase downward, so y is flipped). A shorter focal length gives a wider view, like a wide-angle lens. On the board, W = 640, H = 480 and *f* = 640.



Top-down view. The screen width W stays the same; moving the screen closer to the camera widens the angle it covers.



The ray through one screen pixel

2.3 Rotation

When the sensor turns, the camera turns with it, and every ray turns too. The sensor reports its orientation as three angles:

Angle	Rotation about	Motion
Roll (rx)	the x axis	tilting the head side to side
Pitch (ry)	the y axis	nodding up and down
Yaw (rz)	the z axis	turning left and right

Each rotation is described by a 3 × 3 **rotation matrix**. Writing s_x for sin(rx), c_x for cos(rx), and so on:

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_x & -s_x \\ 0 & s_x & c_x \end{bmatrix} \qquad R_y = \begin{bmatrix} c_y & 0 & s_y \\ 0 & 1 & 0 \\ -s_y & 0 & c_y \end{bmatrix} \qquad R_z = \begin{bmatrix} c_z & -s_z & 0 \\ s_z & c_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The full rotation applies Rx first, then Ry, then Rz. Multiplying the three together gives a single matrix R:

$$R = R_z R_y R_x = \begin{bmatrix} c_y c_z & c_z s_y s_x - s_z c_x & c_z s_y c_x + s_z s_x \\ c_y s_z & s_z s_y s_x + c_z c_x & s_z s_y c_x - c_z s_x \\ -s_y & c_y s_x & c_y c_x \end{bmatrix}$$

R only depends on the orientation, so it is computed **once per frame**. Each ray is then rotated by multiplying R by the ray's vector:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = R \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad \text{so} \qquad \begin{aligned} x' &= r_{00}\,x + r_{01}\,y + r_{02}\,z \\ y' &= r_{10}\,x + r_{11}\,y + r_{12}\,z \\ z' &= r_{20}\,x + r_{21}\,y + r_{22}\,z \end{aligned}$$

Each output element is a **dot product**: three multiplies and two adds. A rotation never changes a vector's length, only its direction.

2.4 From a Direction to a Panorama Pixel

The rotated ray (x′, y′, z′) points in some direction from the center of the sphere. To find which panorama pixel it hits, convert it into longitude and latitude:

$$\text{longitude} = \operatorname{atan2}(x',\, z')$$

$$\text{latitude} = \arcsin\left(\frac{y'}{\sqrt{x'^2 + y'^2 + z'^2}}\right)$$

Longitude is the angle around the vertical axis, measured from straight ahead. Latitude is the angle above or below the horizon. Then scale the two angles to pixel coordinates in the W_p × H_p panorama:

$$x_p = \frac{\text{longitude} + 180^\circ}{360^\circ} \times W_p$$

$$y_p = \frac{90^\circ - \text{latitude}}{180^\circ} \times H_p$$

Adding 180° to the longitude puts straight ahead (0°) at the center column of the panorama. Subtracting the latitude from 90° puts straight up at the top row.

2.5 The Complete Algorithm

Putting it together, rendering one frame takes these steps:

1. **Once per frame:** read the orientation (roll, pitch, yaw) and compute the rotation matrix R.
2. **For every screen pixel,** in raster order (left to right, top to bottom):
   a. Compute the pixel's ray (x, y, z) from its row and column.
   b. Rotate it: (x′, y′, z′) = R × (x, y, z).
   c. Convert the rotated ray to longitude and latitude.
   d. Convert the longitude and latitude to a panorama pixel, and read that pixel's color.
   e. Store the color at the screen pixel's position in the frame buffer.

Each of these steps is one hardware module in the design.

3. Project Setup

You will use **ModelSim** to simulate and verify your design, and **Altera Quartus Prime** to compile, synthesize and program it on the DE25-Standard FPGA development board.

The lab uses this directory structure:

```
Lab 6/
│
├── vhdl/    – VHDL source files for the design.
├── sim/     – ModelSim command file and simulation output.
├── syn/     – Quartus Prime project and constraints.
├── memory/  – The panoramas as memory initialization (.mif) files.
├── images/  – The original panorama images.
├── tools/   – Scripts for loading a panorama into the board's SDRAM.
└── docs/    – Datasheets for the HDMI transmitter and the SDRAM.
```

The design files are:

File	Status	Description
trig_pkg.vhd	Provided	Lookup tables (sine, cos, etc), mult_fixed, and helper functions
rot_matrix.vhd	**TODO**	Computes the rotation matrix R from the three angles
ray_gen.vhd	**TODO**	Generates the ray for every screen pixel, in raster order
mat_vec_mult.vhd	**TODO**	Rotates each ray: multiplies R by the ray vector
vector_to_angles.vhd	Provided	Converts each rotated ray to longitude and latitude
image_reader.vhd	**TODO**	Converts longitude and latitude to a panorama address, reads the pixel from SDRAM, and writes it to the frame buffer
bram.vhd	Provided	Dual-port block RAM, used as the frame buffer
frame_reader.vhd	**TODO**	Reads the frame buffer in raster order for the display
digit_display.vhd	**TODO**	One seven-segment digit
sdram_ctrl.vhd	Provided	SDRAM controller
sdram_model.vhd	Provided	Simulation model of the SDRAM chip
jtag_sdram_writer.vhd	Provided	Receives a panorama over the JTAG cable and writes it to SDRAM
imu.vhd	Provided	Receives and decodes the IMU's orientation data
hdmi.vhd	Provided	640 × 480 video timing, HDMI transmitter pins and I2C setup
vr360.vhd	Provided	Connects the rendering modules, the memories and the displays
vr360_system.vhd	Provided	Board-level top: IMU, renderer, JTAG loader and HDMI output
vr360_tb.vhd	Provided	Testbench that renders five orientations and writes each to a BMP image

There are **three panoramas**, numbered 0, 1 and 2. Each one is provided in three forms:

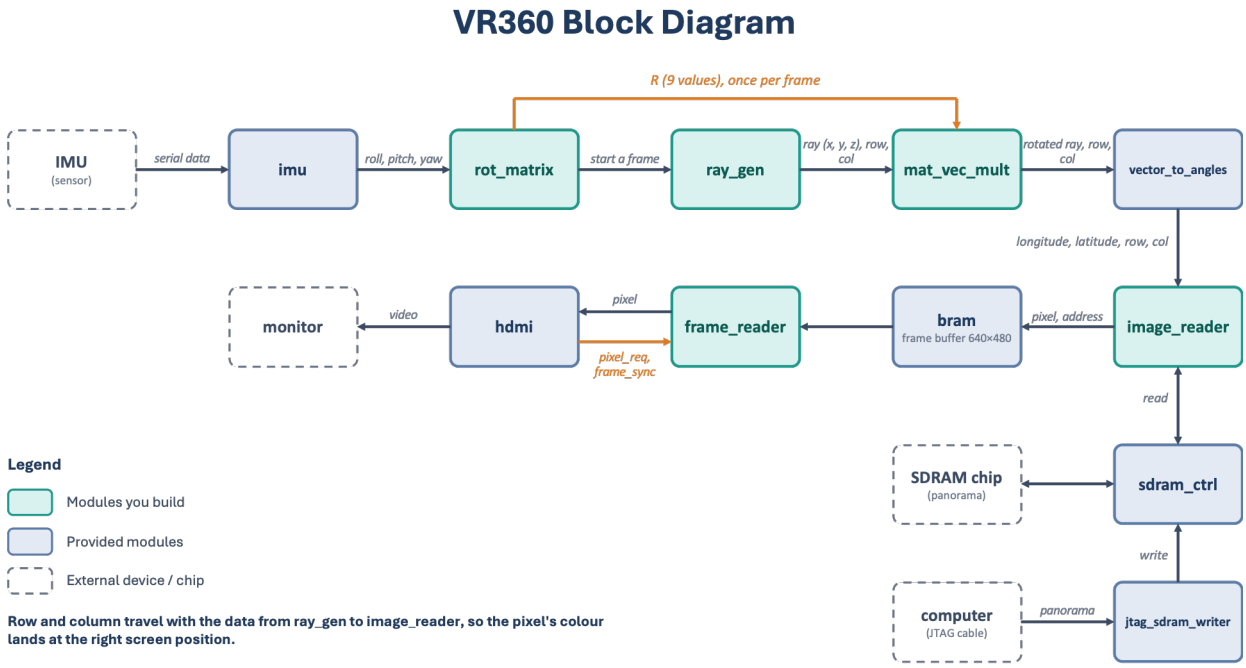
File	Size	Used for
images/vr360_image_N.jpg	Original	Viewing, for comparison with your results
memory/vr360_image_N.mif	4096 × 2048	Loading into the board's SDRAM
memory/vr360_image_N_mini.mif	1024 × 512	Simulation, where a full-size image would be too slow

You will test your design with **all three panoramas**, both in simulation and on the board.

*NOTE: Throughout this assignment, refer to the **DE25-Standard User Manual**, the **ADV7513 HDMI transmitter datasheet** and the **IS42VM16320E SDRAM datasheet** in docs/, and the **Yahboom 9-axis IMU module documentation** (https://github.com/YahboomTechnology/9-axis-IMU-module).*

4. System Design

4.1 Architecture



VR360 Key components:

- **imu** (provided): receives the IMU's serial data and outputs the three orientation angles.
- **rot_matrix:** computes the rotation matrix R from the three angles.
- **ray_gen:** counts through every screen pixel in raster order and outputs its ray, along with its row and column.
- **mat_vec_mult:** rotates each ray by multiplying it by R.
- **vector_to_angles** (provided): converts each rotated ray to longitude and latitude.
- **image_reader:** converts longitude and latitude to a panorama address, reads that pixel from SDRAM, and writes it into the frame buffer at the screen pixel's position.
- **bram** (provided): the **frame buffer**, holding one complete 640 × 480 screen.
- **frame_reader:** reads the frame buffer in raster order, one pixel each time the display asks.

- **hdmi** (provided): generates the 640 × 480, 60 Hz video timing and drives the HDMI transmitter.
- **sdram_ctrl** (provided): the SDRAM controller, which turns simple read and write requests into the commands the SDRAM chip needs.
- **jtag_sdram_writer** (provided): receives a panorama from the computer through the JTAG programming cable and writes it to SDRAM.

Every module from rot_matrix to image_reader passes its row and column along with its data. This way, when image_reader finally has the pixel's color, it knows where on the screen to put it.

4.2 Why the Design Needs a Frame Buffer

The display needs a new pixel every 40 ns, without exception. The renderer cannot keep that pace: each pixel passes through five multi-cycle modules, and the last one waits for the SDRAM, which takes a variable number of clock cycles. Rendering a pixel takes around 15 to 25 clock cycles.

The **frame buffer** solves this. image_reader writes each finished pixel into it whenever it is ready, at address row × 640 + col. Separately, frame_reader reads the pixels out in raster order at the rate the display needs. Neither side ever waits for the other.

Because there is only one frame buffer, the display may show part of the previous frame and part of the new one while rendering is in progress. This visible seam is called **tearing**. A second frame buffer would remove it, at the cost of twice the memory.

Memory sizes. A 640 × 480 screen is 307,200 pixels. The FPGA's on-chip memory is built from **M20K blocks**, each organized as up to 1,024 words of 20 bits, and the device has 358 of them. At 24 bits per pixel, the frame buffer would need about 450 blocks, which does not fit. So vr360.vhd stores each pixel in **20 bits** (6 bits of red, 8 of green and 6 of blue), which takes exactly 300 blocks. Green keeps all 8 bits because the eye is most sensitive to it.

The panorama is far too big for on-chip memory: 4096 × 2048 pixels × 24 bits is about 201 Mbits, against 7.3 Mbits on chip. That is why it is stored in the **external SDRAM** chip on the board.

4.3 Number Formats

The design uses three number formats, all stored in std_logic_vector and interpreted with signed or unsigned from ieee.numeric_std.

Angles: binary angle measure. Angles are 16-bit numbers where the full range represents one complete turn (360°):

$$\text{stored value} = \frac{\text{angle}}{360^\circ} \times 65536$$

Stored value	Angle
0x0000	0°
0x2000	45°
0x4000	90°
0x8000	180° (or −180°)
0xC000	270° (or −90°)

When an angle passes 360°, the 16-bit number simply wraps around to 0, exactly as the angle does. So angle arithmetic needs no special handling for wrap-around. Read as signed, the range is −180° to just under +180°.

Trigonometric values: Q1.15 fixed point. Sin, cos and the rotation matrix elements between −1 and +1. They are stored as 16-bit signed values with **15 fractional bits**, called Q1.15:

$$\text{stored value} = \text{value} \times 2^{15}$$

So 1.0 is stored as 32,767 (the largest positive value, just under 1), 0.5 as 16,384, and −1.0 as −32,768.

Rays: integers. The ray components x, y and z are ordinary signed integers, in pixels.

Fixed-point multiplication. Multiplying two Q1.15 numbers gives a result with 30 fractional bits, so the product must be shifted right by 15 bits to get back to Q1.15. The provided **mult_fixed(a, b)** function in trig_pkg does this for you. It multiplies two 16-bit signed values, rounds, and shifts the product right by 15 bits:

$$\text{mult_fixed}(a, b) = \frac{a \times b}{2^{15}} \quad \text{(rounded)}$$

This works for both kinds of multiply in the design:

- **Q1.15 × Q1.15** (for example, sin × cos in rot_matrix): the result is Q1.15.
- **Q1.15 × integer** (for example, a matrix element × a ray component in mat_vec_mult): dividing by 2^{15} cancels the matrix element's scale factor, so the result is an integer, like the ray.

The result of mult_fixed is already correctly scaled. Do **not** shift it again.

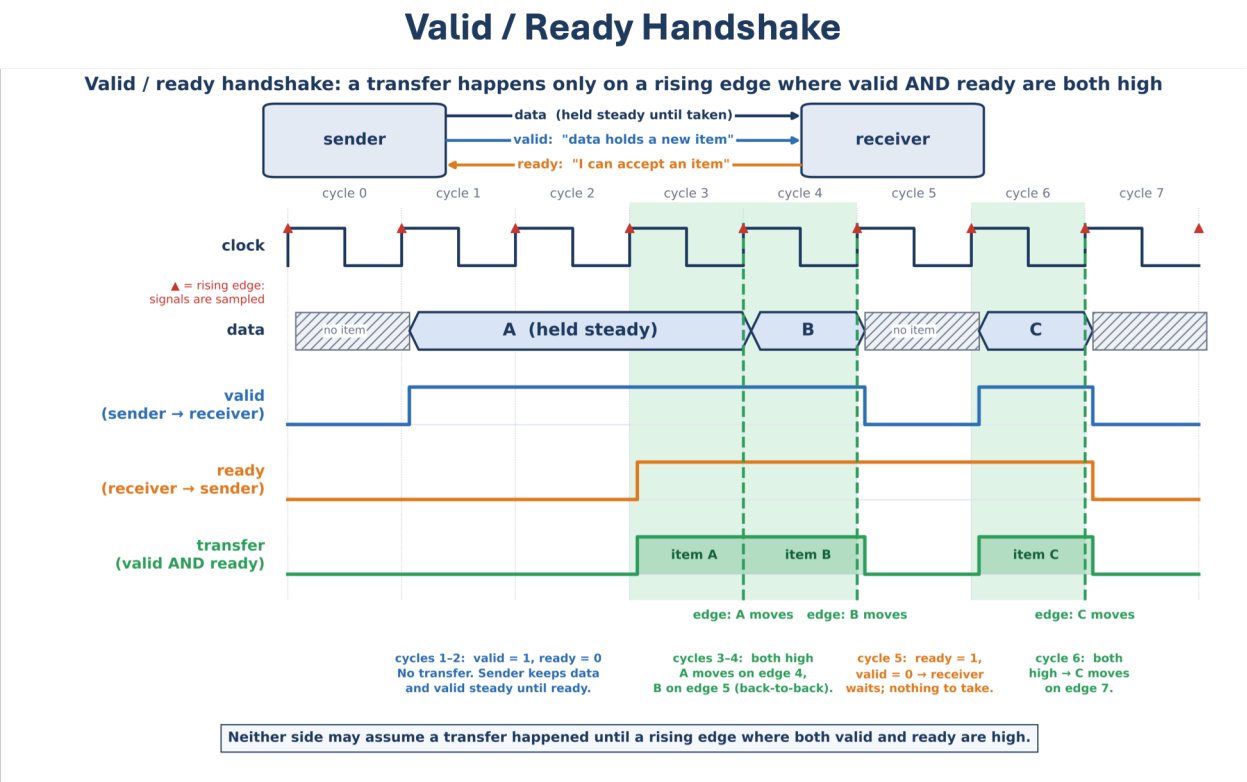
Lookup tables. trig_pkg provides functions that compute trigonometric values with lookup tables (ROMs), since there is no hardware for these functions:

- **sin_lut(angle)** and **cos_lut(angle)** take a 16-bit angle and return its sin or co in Q1.15.
- The arcsine, arctangent, square root and reciprocal tables are used by the provided vector_to_angles.

4.4 The Valid/Ready Handshake

The modules in the chain each take a different number of clock cycles to do their work, and image_reader takes a variable number, since it waits for the SDRAM. So a module cannot assume that the next module is ready for new data, or that the previous one has new data to give. The modules coordinate with a two-way **valid/ready handshake**:

- The **sender** sets **valid** high when its data output holds a new item, and holds the data steady.
- The **receiver** sets **ready** high when it is able to accept a new item.
- An item is **transferred** on a clock edge where **both** valid and ready are high. Neither side may assume a transfer happened until then.



Each module has two sets of handshake signals:

Port	Direction	Meaning
valid_in	in	The previous module has a new item for this module
ready_in	out	This module can accept a new item from the previous module
valid_out	out	This module has a new result for the next module
ready_out	in	The next module can accept a new result

The naming follows the data: valid_in and ready_in belong to the **input** side, and valid_out and ready_out to the **output** side. In vr360.vhd, each module's valid_out connects to the next module's valid_in, and the next module's ready_in connects back to this module's ready_out.

The typical pattern inside a module is:

1. **Wait for input:** set ready_in high. When valid_in is also high, capture the input and start working.
2. **Work:** spend as many clock cycles as needed. Keep ready_in low, so the previous module holds its next item.
3. **Offer the result:** set valid_out high with the result on the output.
4. **Wait for the next module:** when ready_out is high on the same cycle, the result has been taken. Go back to step 1.

If a module's output is not taken yet, it does not accept a new input. This **back-pressure** travels up the chain: when the SDRAM is slow, every module waits, and no data is ever lost or duplicated.

4.5 Two-Process State Machines

Every module you write is a multi-cycle **state machine**, written in the **two-process style** used throughout the provided code:

- **The clocked process** holds every register. On reset, it sets each register to its starting value. On each rising clock edge, it copies each register's **next value** into the register: state <= next_state, x_count <= x_count_c, and so on. It contains no other logic.
- **The combinational process** (process(all)) computes every next value and every output from the current registers and inputs. It has no clock.

Each register therefore has two signals: the register itself (for example x_count) and its next value (x_count_c).

To avoid latches, the combinational process starts by giving **every** next value and output a **default**: each next value defaults to the current register value (x_count_c <= x_count, meaning "keep the value"), and each control output defaults to '0'. The case statement on state then overrides only the values that change in each state. The provided fsm_process skeletons follow this pattern.

The benefit is that each state can be read as "in this state, these are the values the registers take next", and the state machine and its arithmetic are in one place.

4.6 The SDRAM and the JTAG Image Loader

The board's SDRAM chip (IS42VM16320E) holds 64 Mbytes in 16-bit words. Reading it is more involved than reading on-chip memory: the chip must be sent a sequence of commands (open a row, read a column, wait several cycles, close the row), and it must be refreshed regularly or it loses its contents. The provided sdram_ctrl handles all of this. To read a pixel, image_reader uses a simple request handshake:

- Set **rd_addr** to the pixel's address, and hold **rd_req** high.
- Wait until the controller sets **rd_valid** high for one clock cycle. On that cycle, **rd_data** holds the 24-bit pixel.

An FPGA's memory initialization files can only initialize memory inside the FPGA, not an external chip. So the panorama is loaded into the SDRAM **after** the FPGA is configured: the provided **jtag_sdram_writer** receives it over the same USB JTAG cable used for programming, and writes it into the SDRAM pixel by pixel.

4.7 The IMU

The IMU is a **Yahboom 9-axis IMU module**. It combines a 3-axis accelerometer, a 3-axis gyroscope and a 3-axis magnetometer with a small processor that fuses their readings into an orientation, a technique called AHRS (attitude and heading reference system). It reports the orientation as three **Euler angles**: roll, pitch and yaw.

The module sends its data continuously over a **UART** serial link, at **115,200 baud**, about 25 times per second. It needs no commands from the FPGA: only the module's transmit (TX) line is connected, to the FPGA's imu_uart_rx input. The provided imu.vhd:

1. Receives the serial bytes.
2. Finds each message frame: two header bytes (0x7E, 0x23), a length byte, a function code, the data, and a checksum.
3. Keeps only the frames that pass their checksum and carry Euler angles (function code 0x26).
4. Converts the three angles from 32-bit floating-point radians into 16-bit binary angle measure, and outputs them as rx (roll), ry (pitch) and rz (yaw), with a one-cycle data_valid pulse.

4.8 Clocking

The whole design runs on the board's **50 MHz** oscillator, with no PLL. The hdmi module divides it by 2 to get the 25 MHz pixel rate that the 640 × 480, 60 Hz video mode needs. The SDRAM controller sends the SDRAM chip an inverted copy of the same clock. The JTAG cable has its own clock, which Quartus manages inside the JTAG hardware.

5. Design Implementation

In this section, you will complete the six files with **TODO** sections. For each module, you will create a two-process FSM, with one clocked process and one combinational process.

- **digit_display**: Drives one seven-segment digit from a 4-bit value din, showing hexadecimal 0 to F. Six of these show the high byte of each angle: HEX5 and HEX4 for roll, HEX3 and HEX2 for pitch, and HEX1 and HEX0 for yaw.
- **frame_reader**: Reads the frame buffer in raster order by counting the pixels the display has requested. The address resets to 0 while frame_sync is high, increments on each

pixel_req (wrapping after the last pixel), and pixel_valid is pixel_req delayed by one cycle.

- **rot_matrix:** Computes the rotation matrix R once per frame. To meet timing at 50 MHz, the work is split across multiple FSM states: look up the sines and cosines, form the two-term products, form the three-term products, then assemble the nine matrix elements.
- **ray_gen**: Once per frame, steps through every screen pixel in raster order and outputs its ray along with its row and column. It waits for a new frame, outputs one ray, then advances the column and row counters, returning to the wait state after the last pixel.
- **mat_vec_mult**: Rotates each ray by R. It computes all nine products in one cycle, then adds each row's three products in the next, passing the row and column along with the result.
- **image_reader**: Converts each longitude and latitude to a panorama address, reads that pixel from SDRAM, and writes it into the frame buffer at the screen position the ray came from. The first state computes the address, and the second waits for the SDRAM to return the pixel and then writes it.

6. Simulation

Before programming the FPGA, verify the design in ModelSim. The testbench simulates vr360, with the provided SDRAM model standing in for the SDRAM chip. It loads a panorama into the model, renders it from **5 orientations**, and writes each frame to a BMP image you can open.

6.1 The Testbench

The provided **vr360_tb** uses smaller settings than the board so that the simulation finishes in a reasonable time. Each output pixel needs a full SDRAM access, so a 640 × 480 frame would take over an hour to simulate.

Setting	Board	Testbench
Panorama	4096 × 2048	1024 × 512 (_mini.mif)
Screen	640 × 480	160 × 120
Focal length	640	80

The focal length is scaled with the screen width so that the field of view stays the same. The output is a small version of the same view, not a crop of it. The testbench renders these five orientations, in degrees:

Output file	Roll (rx)	Pitch (ry)	Yaw (rz)
vr360_pose_0.bmp	0	0	0
vr360_pose_1.bmp	30	0	0
vr360_pose_2.bmp	0	30	0
vr360_pose_3.bmp	0	0	30
vr360_pose_4.bmp	−30	30	−20

6.2 Create the ModelSim Command File

Complete sim/sim.do. It should compile the files in dependency order, start the testbench, add the important signals to the waveform viewer, and run the simulation. Do not compile vr360_system.vhd, jtag_sdram_writer.vhd, imu.vhd or hdmi.vhd: the JTAG writer uses a hardware block that has no simulation model, so the system level cannot be simulated.

```
vlib work

vcom -2008 ../vhdl/trig_pkg.vhd
vcom -2008 ../vhdl/bram.vhd
vcom -2008 ../vhdl/digit_display.vhd
vcom -2008 ../vhdl/rot_matrix.vhd
vcom -2008 ../vhdl/ray_gen.vhd
vcom -2008 ../vhdl/mat_vec_mult.vhd
vcom -2008 ../vhdl/vector_to_angles.vhd
vcom -2008 ../vhdl/image_reader.vhd
vcom -2008 ../vhdl/frame_reader.vhd
vcom -2008 ../vhdl/sdram_ctrl.vhd
vcom -2008 ../vhdl/sdram_model.vhd
vcom -2008 ../vhdl/vr360.vhd
vcom -2008 ../vhdl/vr360_tb.vhd

vsim -voptargs=+acc vr360_tb

add wave -group "TESTBENCH"        sim:/vr360_tb/*
add wave -group "VR360"            sim:/vr360_tb/dut/*
add wave -group "ROTATION MATRIX"  sim:/vr360_tb/dut/u_rotation_matrix/*
add wave -group "RAY GENERATOR"    sim:/vr360_tb/dut/u_ray_gen/*
add wave -group "MAT VEC MULTIPLY" sim:/vr360_tb/dut/u_mat_vec_mult/*
add wave -group "VECTOR TO ANGLES" sim:/vr360_tb/dut/u_vector_to_angles/*
add wave -group "IMAGE READER"     sim:/vr360_tb/dut/u_image_reader/*
add wave -group "FRAME READER"     sim:/vr360_tb/dut/u_frame_reader/*

run -all
```

6.3 Run the Simulation for Each Panorama

Open ModelSim, change to the sim directory, and run:

```
do sim.do
```

Each pose takes several milliseconds of simulated time, so the run takes a few minutes. Then repeat the simulation for **each of the three panoramas**: change IMAGE_FILE to vr360_image_1_mini.mif, then vr360_image_2_mini.mif, and run it again. Each run overwrites the five output images, so rename or move them to a separate folder after each run.

6.4 Verify the Results

For each panorama, open the five output images next to the original vr360_image_N.jpg, and verify that **No image is shifted, wrapped, striped or noisy.**

In the waveform viewer, check the handshakes: on every clock edge where a module's valid_out and the next module's ready_in are both high, exactly one item should move forward.





R_x=0, R_y=0, R_z=0



R_x=30, R_y=0, R_z=0



R_x=0, R_y=30, R_z=0



R_x=0, R_y=0, R_z=30



R_x=-30, R_y=30, R_z=-20

7. Synthesis

Once the design simulates correctly, implement it on the DE25-Standard.

7.1 Open the Quartus Prime Project

Open the provided project syn/vr360.qpf in Quartus Prime. It is already set up with:

- Top-level entity: **vr360_system**
- Device: Agilex 5, **A5ED013BB32AE4SCS**
- The timing constraints file, **vr360.sdc**

Add all the VHDL files from the vhdl directory to the project, **except** for sdram_model.vhd and vr360_tb.vhd, as they are for simulation only.

7.2 Timing Constraints

The provided syn/vr360.sdc is complete. Read through it and understand what each constraint does:

- **create_clock -period "50MHz" [get_ports clk_50]** defines the 50 MHz board clock, which drives the whole design.
- **derive_clock_uncertainty** adds timing margins for clock jitter.
- **set_false_path** on the reset button, the seven-segment displays, the LEDs, the IMU serial input and the HDMI outputs. None of these have a meaningful single-clock timing

requirement: the button and the IMU line change at random times, the displays and LEDs are only seen by a person, and the HDMI clock and data are generated together by the same logic, so they stay aligned.

- **SDRAM interface constraints.** A **virtual clock** (sdram_clk_virt) represents the clock at the SDRAM chip. **set_input_delay** and **set_output_delay** describe, from the SDRAM datasheet, how long the chip takes to drive data back and how much setup and hold time it needs. These let the Timing Analyzer check that signals reach the chip, and data reaches the FPGA, in time.
- **JTAG clock.** The block at the end creates the JTAG cable's clock and declares it **asynchronous** to the system clock with set_clock_groups -asynchronous, since the two have no timing relationship.

7.3 Assign FPGA Pins

Open **Assignments → Pin Planner** and assign the following signals:

Port	FPGA Pin	I/O Standard	Description
clk_50	BM71	1.2-V	50 MHz board clock
reset_n	BM78	1.2-V	CPU reset button
imu_uart_rx	BU28	3.3-V LVCMOS	IMU serial data (from the module's TX), on the GPIO header
leds[0]	CC71	1.2-V	LED 0: IMU serial line
leds[1]	BH78	1.2-V	LED 1: IMU bytes received
leds[2]	CH69	1.2-V	LED 2: IMU frames accepted
leds[3]	CF69	1.2-V	LED 3: panorama loading
leds[4]	CA62	1.2-V	LED 4: design running (blinks)
hex0[0]	BP81	1.2-V	HEX0 segment a
hex0[1]	BM81	1.2-V	HEX0 segment b
hex0[2]	BM89	1.2-V	HEX0 segment c
hex0[3]	BK89	1.2-V	HEX0 segment d
hex0[4]	BH92	1.2-V	HEX0 segment e
hex0[5]	BM92	1.2-V	HEX0 segment f
hex0[6]	BP92	1.2-V	HEX0 segment g
hex1[0]	CA78	1.2-V	HEX1 segment a
hex1[1]	BW78	1.2-V	HEX1 segment b
hex1[2]	BU78	1.2-V	HEX1 segment c
hex1[3]	BR78	1.2-V	HEX1 segment d
hex1[4]	BU81	1.2-V	HEX1 segment e
hex1[5]	BR81	1.2-V	HEX1 segment f
hex1[6]	CA89	1.2-V	HEX1 segment g
hex2[0]	BW89	1.2-V	HEX2 segment a

Port	FPGA Pin	I/O Standard	Description
hex2[1]	BU92	1.2-V	HEX2 segment b
hex2[2]	BR92	1.2-V	HEX2 segment c
hex2[3]	BU89	1.2-V	HEX2 segment d
hex2[4]	BR89	1.2-V	HEX2 segment e
hex2[5]	CF78	1.2-V	HEX2 segment f
hex2[6]	CH78	1.2-V	HEX2 segment g
hex3[0]	CC81	1.2-V	HEX3 segment a
hex3[1]	CA81	1.2-V	HEX3 segment b
hex3[2]	CH81	1.2-V	HEX3 segment c
hex3[3]	CF81	1.2-V	HEX3 segment d
hex3[4]	CF89	1.2-V	HEX3 segment e
hex3[5]	CH89	1.2-V	HEX3 segment f
hex3[6]	CH92	1.2-V	HEX3 segment g
hex4[0]	CF92	1.2-V	HEX4 segment a
hex4[1]	CA92	1.2-V	HEX4 segment b
hex4[2]	CC92	1.2-V	HEX4 segment c
hex4[3]	CL76	1.2-V	HEX4 segment d
hex4[4]	CK76	1.2-V	HEX4 segment e
hex4[5]	CL82	1.2-V	HEX4 segment f
hex4[6]	CK80	1.2-V	HEX4 segment g
hex5[0]	CL85	1.2-V	HEX5 segment a
hex5[1]	CK85	1.2-V	HEX5 segment b
hex5[2]	BK69	1.2-V	HEX5 segment c
hex5[3]	BP71	1.2-V	HEX5 segment d
hex5[4]	BH89	1.2-V	HEX5 segment e
hex5[5]	BH71	1.2-V	HEX5 segment f
hex5[6]	CH62	1.2-V	HEX5 segment g
dram_clk	B39	1.8-V LVCMOS	SDRAM clock
dram_cke	B35	1.8-V LVCMOS	SDRAM clock enable
dram_cs_n	H8	1.8-V LVCMOS	SDRAM chip select (active low)
dram_ras_n	G1	1.8-V LVCMOS	SDRAM row address strobe (active low)
dram_cas_n	J1	1.8-V LVCMOS	SDRAM column address strobe (active low)
dram_we_n	J2	1.8-V LVCMOS	SDRAM write enable (active low)
dram_ldqm	K4	1.8-V LVCMOS	SDRAM lower byte mask
dram_udqm	G2	1.8-V LVCMOS	SDRAM upper byte mask
dram_ba[0]	D34	1.8-V LVCMOS	SDRAM bank address bit 0
dram_ba[1]	F4	1.8-V LVCMOS	SDRAM bank address bit 1

Port	FPGA Pin	I/O Standard	Description
dram_addr[0]	F27	1.8-V LVCMOS	SDRAM address bit 0
dram_addr[1]	F24	1.8-V LVCMOS	SDRAM address bit 1
dram_addr[2]	H27	1.8-V LVCMOS	SDRAM address bit 2
dram_addr[3]	D24	1.8-V LVCMOS	SDRAM address bit 3
dram_addr[4]	H18	1.8-V LVCMOS	SDRAM address bit 4
dram_addr[5]	D15	1.8-V LVCMOS	SDRAM address bit 5
dram_addr[6]	F18	1.8-V LVCMOS	SDRAM address bit 6
dram_addr[7]	F15	1.8-V LVCMOS	SDRAM address bit 7
dram_addr[8]	K8	1.8-V LVCMOS	SDRAM address bit 8
dram_addr[9]	F8	1.8-V LVCMOS	SDRAM address bit 9
dram_addr[10]	C2	1.8-V LVCMOS	SDRAM address bit 10
dram_addr[11]	A17	1.8-V LVCMOS	SDRAM address bit 11
dram_addr[12]	D4	1.8-V LVCMOS	SDRAM address bit 12
dram_dq[0]	A14	1.8-V LVCMOS	SDRAM data bit 0
dram_dq[1]	A11	1.8-V LVCMOS	SDRAM data bit 1
dram_dq[2]	B14	1.8-V LVCMOS	SDRAM data bit 2
dram_dq[3]	B11	1.8-V LVCMOS	SDRAM data bit 3
dram_dq[4]	A20	1.8-V LVCMOS	SDRAM data bit 4
dram_dq[5]	A8	1.8-V LVCMOS	SDRAM data bit 5
dram_dq[6]	A23	1.8-V LVCMOS	SDRAM data bit 6
dram_dq[7]	B4	1.8-V LVCMOS	SDRAM data bit 7
dram_dq[8]	B20	1.8-V LVCMOS	SDRAM data bit 8
dram_dq[9]	B23	1.8-V LVCMOS	SDRAM data bit 9
dram_dq[10]	B26	1.8-V LVCMOS	SDRAM data bit 10
dram_dq[11]	B30	1.8-V LVCMOS	SDRAM data bit 11
dram_dq[12]	A39	1.8-V LVCMOS	SDRAM data bit 12
dram_dq[13]	A30	1.8-V LVCMOS	SDRAM data bit 13
dram_dq[14]	A35	1.8-V LVCMOS	SDRAM data bit 14
dram_dq[15]	A33	1.8-V LVCMOS	SDRAM data bit 15
hdmi_tx_clk	CH59	1.2-V	HDMI transmitter pixel clock
hdmi_tx_hs	BR109	3.3-V LVCMOS	HDMI horizontal sync
hdmi_tx_vs	BE107	3.3-V LVCMOS	HDMI vertical sync
hdmi_tx_de	BK109	3.3-V LVCMOS	HDMI data enable
hdmi_tx_d[0]	CD134	3.3-V LVCMOS	HDMI pixel data bit 0
hdmi_tx_d[1]	CD135	3.3-V LVCMOS	HDMI pixel data bit 1
hdmi_tx_d[2]	CG134	3.3-V LVCMOS	HDMI pixel data bit 2
hdmi_tx_d[3]	CG135	3.3-V LVCMOS	HDMI pixel data bit 3

Port	FPGA Pin	I/O Standard	Description
hdmi_tx_d[4]	CH132	3.3-V LVCMOS	HDMI pixel data bit 4
hdmi_tx_d[5]	CF132	3.3-V LVCMOS	HDMI pixel data bit 5
hdmi_tx_d[6]	CF128	3.3-V LVCMOS	HDMI pixel data bit 6
hdmi_tx_d[7]	CK134	3.3-V LVCMOS	HDMI pixel data bit 7
hdmi_tx_d[8]	CL125	3.3-V LVCMOS	HDMI pixel data bit 8
hdmi_tx_d[9]	CF121	3.3-V LVCMOS	HDMI pixel data bit 9
hdmi_tx_d[10]	CF118	3.3-V LVCMOS	HDMI pixel data bit 10
hdmi_tx_d[11]	BU118	3.3-V LVCMOS	HDMI pixel data bit 11
hdmi_tx_d[12]	BR118	3.3-V LVCMOS	HDMI pixel data bit 12
hdmi_tx_d[13]	CA118	3.3-V LVCMOS	HDMI pixel data bit 13
hdmi_tx_d[14]	BW118	3.3-V LVCMOS	HDMI pixel data bit 14
hdmi_tx_d[15]	CL128	3.3-V LVCMOS	HDMI pixel data bit 15
hdmi_tx_d[16]	CL130	3.3-V LVCMOS	HDMI pixel data bit 16
hdmi_tx_d[17]	CK125	3.3-V LVCMOS	HDMI pixel data bit 17
hdmi_tx_d[18]	CK128	3.3-V LVCMOS	HDMI pixel data bit 18
hdmi_tx_d[19]	BF111	3.3-V LVCMOS	HDMI pixel data bit 19
hdmi_tx_d[20]	BH109	3.3-V LVCMOS	HDMI pixel data bit 20
hdmi_tx_d[21]	BE115	3.3-V LVCMOS	HDMI pixel data bit 21
hdmi_tx_d[22]	BF115	3.3-V LVCMOS	HDMI pixel data bit 22
hdmi_tx_d[23]	BU109	3.3-V LVCMOS	HDMI pixel data bit 23
hdmi_tx_int	BE111	3.3-V LVCMOS	HDMI transmitter interrupt (active low)
hdmi_i2c_scl	BF120	3.3-V LVCMOS	HDMI transmitter I2C clock
hdmi_i2c_sda	BH118	3.3-V LVCMOS	HDMI transmitter I2C data

7.4 Compile and Synthesize

Run the complete Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation reports and verify that:

- **vr360_system** is the top-level entity, and all top-level ports have pin assignments.
- **Memory:** the Fitter's resource usage summary shows about **300 M20K blocks** for the frame buffer, plus a few more for the lookup tables. If the frame buffer is built from logic instead, the design will not fit.
- **DSP blocks:** the multipliers in rot_matrix, mat_vec_mult and vector_to_angles use DSP blocks. Note how many your design uses.
- **Timing:** the Timing Analyzer shows the 50 MHz clock with no setup or hold violations. If rot_matrix or mat_vec_mult fails timing, check that each state does only one level of multiplication, as described in Section 5.

8. Hardware Setup and Testing

8.1 The IMU Module

The IMU is the **Yahboom 9-axis IMU sensor module**, with AHRS orientation output. For reference, its documentation, wiring diagrams and configuration software are at https://github.com/YahboomTechnology/9-axis-IMU-module.

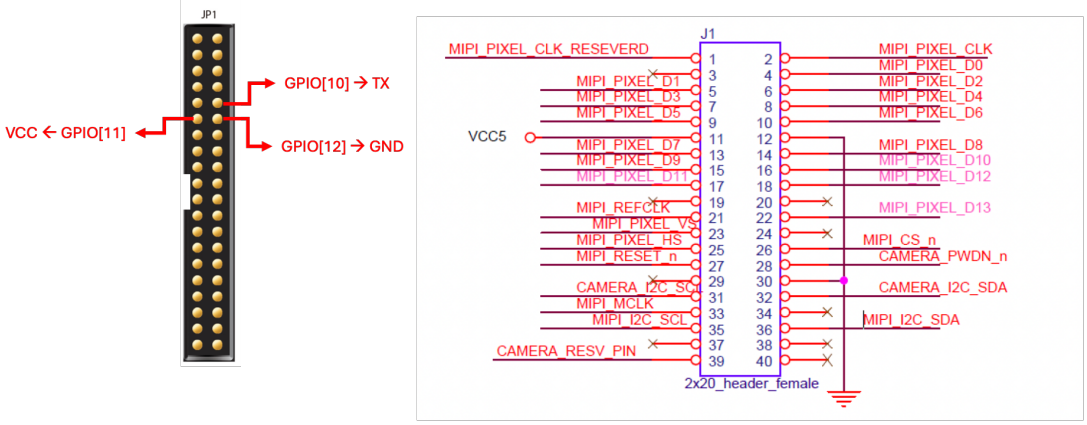
Note: You will need to solder the pins for the serial communication ports. Please make sure the pins are properly connected and do not touch each other, or you may burn out the chip.

Default settings. The module must be set to stream **Euler angle** data at **115,200 baud**, which is how imu.vhd expects it. These are the module's defaults.

Calibration. The orientation is only as good as the sensor's calibration. If the angles drift or jump, follow the module documentation to calibrate the accelerometer, gyroscope and magnetometer. Keep the module away from magnets, motors and large metal objects, which disturb the magnetometer and therefore the yaw angle.

Wiring. Make all connections with both the board and the IMU module powered off. Only three wires are needed (RX is not required):

IMU module	Pin	DE25-Standard
TX (serial data out)	GPIO[10]	imu_uart_rx pin on the GPIO header
VCC	GPIO[11]	5V supply voltage pin on the GPIO header
GND	GPIO[12]	GND pin on the GPIO header



8.2 Connect the Board

Make all connections with the FPGA board powered off.

- Connect the IMU module as described in Section 8.1.
- Connect a monitor to the board's **HDMI** port.
- Connect the board to the computer through its **USB-Blaster** connection.

8.3 Program the FPGA and Load a Panorama

The panorama must be loaded into the SDRAM **after** the FPGA is programmed, using the **tools/send_image.sh** script. Before running it, open it and set QUARTUS_BIN to the bin64 folder of your Quartus installation.

1. Power on the board and the monitor.
2. **Program the FPGA**, either with **Tools → Programmer** in Quartus Prime
3. **Load the test pattern** into the SDRAM: `./send_image.sh`
4. **Load a panorama** into the SDRAM: `./send_image.sh ../memory/vr360_image_0.mif`

The image is sent over the JTAG cable in blocks of pixels, which takes a minute or two. LED 3 stays lit while the image is arriving, and the picture on the monitor fills in as it loads.

The SDRAM keeps the panorama until the board loses power or is reprogrammed. To switch to a different panorama, run step 3 again with another file. You do not need to reprogram the FPGA.

8.4 Test the System

Check the status LEDs first. They show whether each part of the system is working:

LED	Meaning
LED 0	The raw IMU serial line. Dark or faintly flickering means the line is healthy. Solid means the line is stuck.
LED 1	Flickers while serial bytes are received. Dark with a healthy line usually means the wrong baud rate.
LED 2	Blinks for each accepted orientation frame. Dark while LED 1 flickers means bytes arrive but no frame passes its checksum, so the module is probably sending a different message format.
LED 3	Lit while a panorama is being loaded.
LED 4	Blinks about once per second while the design is running. If it is dark or solid, the design is not running.

Check the angles. The seven-segment displays show the high byte of each angle in hexadecimal: HEX5 and HEX4 show roll, HEX3 and HEX2 pitch, and HEX1 and HEX0 yaw. A full turn is 00 to FF, so 40 is 90°, 80 is 180° and C0 is 270° (or −90°). Turn the IMU about each axis and verify that the matching pair of digits changes.

Test the viewer with each of the **three panoramas**, loading each one in turn:

- With the IMU level and facing forward, verify that the monitor shows an undistorted view of the panorama.
- Turn the IMU left and right (yaw), and verify that the view turns smoothly all the way around, with no seam or jump at ±180°.
- Tilt it up and down (pitch), and verify that you can look up to the sky and down to the ground.
- Roll it side to side, and verify that the horizon tilts.
- Compare the view at a known orientation with the matching simulation image.
- Check that as you rotate the IMU in each direction, the HEX display changes as well.

9. Submission

Zip your folder, name it **CE355_Lab6_Team_[N].zip**, and submit it to Canvas. Make sure to clean out any synthesis and simulation work directories, and do not include the generated BMP files.

9.1 Report

Submit a report of no more than 2–3 pages, figures included. Be concise: every figure must be referenced and explained in the text, and must be legible — crop and zoom rather than pasting a full-desktop screenshot. Cover the following.

- **Simulation.** The transcript output from the simulator log window, including one or two annotated waveforms and an explanation of how you know the design is working. Show at least one valid/ready handshake between modules. Explain the test cases you ran, and include the images rendered in simulation for each panorama.
- **Schematic.** An RTL Viewer image showing your structure and correct connections, including instantiation of key components — the frame buffer block RAM, the SDRAM controller, the DSP multipliers and any IP. Note anything the tool inferred that you did not intend: a latch, LUT-based memory instead of a block RAM, a multiplier built from logic.
- **Resources.** Based on the synthesis results, determine whether the resources used align with what you expect: the number of multipliers (DSP blocks) in rot_matrix, mat_vec_mult and vector_to_angles, the memory bits and blocks used by the frame buffer, the approximate register count, and any other IP. Explain any significant differences.
- **Timing.** fMAX and worst-case setup slack for each clock, the registers the critical path runs between, and why it is the slowest path. “Timing met” is not an analysis. Explain any changes you made to your design to meet timing if it was not met initially.
- **Optimizations.** Discuss any optimizations or techniques you implemented and their significance.
- **Hardware.** Which hardware tests passed, and any differences between simulation and hardware.
- **Screenshots** of the design working correctly, both in simulation and on the board. List approximations of the pitch, roll and yaw corresponding to each image.

9.2 Demonstration

You will also need to demo the hardware setup and simulation during TA office hours. Be prepared to explain how the system works, including how a pixel becomes a ray, how the rotation matrix is built and why it is split across states, how mult_fixed scales its result, how the valid/ready handshake prevents lost or duplicated data, how longitude and latitude become a panorama address, why the design needs both an SDRAM and a frame buffer, and how the IMU’s data reaches the renderer.